

```

import json
import pandas as pd

file_path = "/content/unified_gacm_dataset.json"

# Step 1: Open and parse the JSON file safely using python's json module
with open(file_path, "r", encoding="utf-8") as f:
    data = json.load(f)

# Step 2: Use json_normalize to automatically flatten the nested dictionary structure
# If 'data' is wrapped inside a main key (e.g. {'records': [...]}), change it to: pd.json_normalize(data['records'])
if isinstance(data, dict) and len(data) == 1:
    # If nested under a single main root key, extract that list automatically
    root_key = list(data.keys())[0]
    df = pd.json_normalize(data[root_key])
else:
    df = pd.json_normalize(data)

# Step 3: View the dataset overview
print("--- FIRST 5 ROWS (HEAD) ---")
display(df.head())

print("\n--- DATASET INFORMATION (INFO) ---")
df.info()

```

--- FIRST 5 ROWS (HEAD) ---

	version	description	created_at	total_records	source_systems	records
0	1.0.0	Unified Enterprise Master Dataset for GACM Gra...	2026-08-29T12:31:39.463550+00:00	15051	[NSF_Grants, Institutional_Manifests, MISEd_Me...	{'record_id': 'grant_5900035', 'external_id':...

--- DATASET INFORMATION (INFO) ---

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 1 entries, 0 to 0

Data columns (total 6 columns):

#	Column	Non-Null Count	Dtype
0	version	1 non-null	object
1	description	1 non-null	object
2	created_at	1 non-null	object
3	total_records	1 non-null	int64
4	source_systems	1 non-null	object
5	records	1 non-null	object

dtypes: int64(1), object(5)

memory usage: 180.0+ bytes

```

import json
import pandas as pd

file_path = "/content/unified_gacm_dataset.json"

# Load raw JSON data
with open(file_path, "r", encoding="utf-8") as f:
    data = json.load(f)

# FIX: Flatten the actual list inside the 'records' key
df = pd.json_normalize(data["records"])

print("--- NEW DATASET INFORMATION (INFO) ---")
df.info()

print("\n" + "=" * 50 + "\n")

# Dynamically search for any column name containing 'id'
id_columns = [col for col in df.columns if "id" in col.lower()]

if id_columns:
    # Use the first ID column found
    target_col = id_columns[0]
    print(f"Found ID column: '{target_col}'")

    total_records = df[target_col].count()
    unique_records = df[target_col].nunique()
    missing_records = df[target_col].isna().sum()

    print(f"Total entries: {total_records}")
    print(f"Unique entries: {unique_records}")
    print(f"Missing (Null) values: {missing_records}")

    if total_records != unique_records:

```

```

    print(f"⚠ Alert: There are {total_records - unique_records} duplicates!")
else:
    print("✅ All entries are unique.")
else:
    print("❌ Still no 'id' column found inside the records.")
    print("Available columns inside records are:", df.columns.tolist()[:10])

```

```

2  source_system          15051 non-null object
3  connector_type         15051 non-null object
4  record_type            15051 non-null object
5  institution_id         15051 non-null object
6  title                  15051 non-null object
7  content                15051 non-null object
8  graph_triples          15051 non-null object
9  tags                   15051 non-null object
10 author.id              15051 non-null object
11 author.name            15051 non-null object
12 author.role            15051 non-null object
13 organization.id        15051 non-null object
14 organization.name       15051 non-null object
15 metadata.grant_id       10000 non-null object
16 metadata.award_amount   10000 non-null float64
17 metadata.start_date     10000 non-null object
18 raw_data.grant_id       10000 non-null object
19 raw_data.project_title  10000 non-null object
20 raw_data.faculty_name   10000 non-null object
21 raw_data.institution    12129 non-null object
22 raw_data.award_amount   10000 non-null object
23 raw_data.start_date     10000 non-null object
24 raw_data.abstract       10000 non-null object
25 metadata.governing_body 2129 non-null object
26 metadata.meeting_date   2129 non-null object
27 metadata.date_confidence 2129 non-null object
28 metadata.source_url     2129 non-null object
29 metadata.document_type  2129 non-null object
30 metadata.segment_index  2129 non-null float64
31 raw_data.governing_body 2129 non-null object
32 raw_data.meeting_date   2129 non-null object
33 raw_data.date_confidence 2129 non-null object
34 raw_data.source_url     2129 non-null object
35 raw_data.parent_source_url 1936 non-null object
36 raw_data.document_type  2129 non-null object
37 raw_data.segment_index  2129 non-null float64
38 raw_data.segment_title  2129 non-null object
39 raw_data.segment_text   2129 non-null object
40 metadata.meeting_id     2922 non-null object
41 metadata.dialog_id      2922 non-null object
42 metadata.turn_index     2922 non-null float64
43 metadata.query_type     2922 non-null object
44 raw_data.query          2922 non-null object
45 raw_data.response       2922 non-null object
46 raw_data.responseAttribution.indexRanges 1980 non-null object
47 raw_data.queryMetadata.queryType 2922 non-null object
48 raw_data.queryMetadata.isUnanswerable 927 non-null object
49 raw_data.queryMetadata.isContextDependent 388 non-null object
dtypes: float64(4), object(46)
memory usage: 5.7+ MB

```

```
=====
```

```

Found ID column: 'record_id'
Total entries: 15051
Unique entries: 13687
Missing (Null) values: 0
⚠ Alert: There are 1364 duplicates!

```

